

דוח טכני: שליטה ברכב רובוטי באמצעות זיהוי שפת סימנים (ASL)

בזמן (ASL) שם הפרויקט: שליטה ברכב רובוטי באמצעות זיהוי תנועות שפת הסימנים האמריקאית
אמת

שמות הסטודנטים: ליאל אווננו, סהר באביאן

תאריך הגשה: 13.7.26

1. תקציר

הפרויקט משלב למידה עמוקה עם רובוטיקה פיזית: רכב אוטונומי הנשלט בתנועות יד בשפת המערכת בנויה משני בקרים — מחשב נייד עם מצלמה, המריץ רשת (ASL) הסימנים האמריקאית וסרוו היגוי. DC המפעיל רכב עם שני מנועי **Arduino Mega** ובקר, TinyML בטכנולוגיית CNN ועברה כוונן עדין (תמונות אימון 27,455) Sign Language MNIST הרשת אומנה מאפס על מאגר לצורך int8- עם דגימות שנלכדו ממצלמת הפרויקט, ולאחר מכן קוונטיזציה מלאה ל (Fine-tuning) (W, A, C, B, O) הטמעה על המיקרו-בקר. המערכת מזהה 24 אותיות, ומתרגמת חמש מהן (Majority Vote) לפקודות נסיעה — קדימה, אחורה, שמאלה, ימינה ועצירה — לאחר החלקת-הצבעה על מספר פריימים וסף ביטחון של 80%, עם עצירת בטיחות אוטומטית לאחר 1.5 שניות ללא (Majority Vote) פקודה.

2. מבוא

2.1 הגדרת הבעיה

ASL המשימה: לאפשר שליטה ברכב רובוטי פיזי באמצעות תנועות יד בלבד. המשתמש מציג אות למצלמת המחשב, והמערכת מזהה אותה ומתרגמת לפקודת נסיעה בזמן אמת. האתגר המרכזי הוא ביצוע סיווג תמונה על חומרה פנימי בלבד — תוך שמירה על דיוק מספק לשליטה אמינה.

2.2 מוטיבציה וגישה

רשת נוירונים נבחרה מכיוון שגישות קלאסיות של עיבוד תמונה (התאמת תבניות, סף צבע) אינן ולאחריו (Training from Scratch) עמידות לשינוי תאורה, רקע וזווית. סוג השימוש: אימון מאפס עם דגימות מותאמות. הסיבות Fine-tuning:

- מודל קטן מטבעו — קלט 28×28 בגווי אפור מאפשר
- איסוף דגימות מהמצלמה הספציפית של — (Domain Shift) סוגר פער דומיין Fine-tuning הפרויקט ואימון מחדש מתאימים את המודל לתנאי השטח, לעומת תמונות הסטודיו הנקיות

3. חומרה, סביבה ורקע טכנולוגי

3.1 הפלטפורמה הרובוטית

רכיב	תיאור
מיקרו-בקר זיהוי	Xtensa LX7 Dual-Core 240MHz, פנימי 8MB PSRAM, ~255KB DRAM מחשב נייד
מצלמה	מחשב נייד, לכידה ב-96×96 גווני אפור
מיקרו-בקר רכב	Arduino Mega 2560 — ATmega2560, 16MHz
מנועים	עם אינקודרים (פינים 2/3, 18/19), DC מנועי 2 L298N דרייבר
היגוי	סרוו על פין 9 (30°=ימין, 50°=ישר, 70°=שמאל)
תקשורת בין הבקרים	Serial1 (GPIO2) → RX2 (פין 17), 9,600 baud

המצלמה היא חיישן הקלט היחיד. הפריים (96×96, גווני אפור) עובר חיתוך מרכזי (70% מהפריים) ל-28×28 (Nearest-Neighbor לא, Box-Average Downsample) ולאחר מכן דגימה-מחדש בממוצע-תיבה 28 פיקסלים — גודל קלט המודל. אין זיהוי-יד אמיתי על המיקרו-בקר (יקר מדי חישובית); המשתמש נדרש למרכז את היד בפריים.

3.2 סביבת האימון

OpenCV עם tensorflow/tensorflow:2.15.0 מבוסס Docker אימון מתבצע בקונטיינר הסביבה רצה (C++ Header-המרה ל-xxd ו-CSV טעינת) NumPy, Pandas, (לכידה ועיבוד תמונה) בזמן אימון ובדיקה OpenCV להצגת חלונות X11 ול-/dev/video0 עם גישה ל Ubuntu על

3.3 מותאם CNN — המודל שנבחר

קטנה CNN אומנה כאן רשת (YOLO, MobileNet, ResNet) בניגוד לשימוש במודל פרה-מאומן גדול מאפס, מכיוון ש

- הקלט (28×28 גווני אפור, 784 ערכים) פשוט ואינו דורש ארכיטקטורה עמוקה.
- מחלקות בלבד — מספר קטן שמאפשר לרשת קלה להשיג דיוק גבוה 24.

Layer (type)	Output Shape	Params
Input	(28, 28, 1)	0
Conv2D (75 filters, 3×3)	(28, 28, 75)	750
BatchNormalization	(28, 28, 75)	300
MaxPool2D (2×2)	(14, 14, 75)	0
Conv2D (100 filters, 3×3)	(14, 14, 100)	67,600
Dropout (0.2)	(14, 14, 100)	0
BatchNormalization	(14, 14, 100)	400

MaxPool2D (2×2)	(7, 7, 100)	0
Conv2D (128 filters, 3×3)	(7, 7, 128)	115,328
Dropout (0.3)	(7, 7, 128)	0
BatchNormalization	(7, 7, 128)	512
MaxPool2D (2×2)	(4, 4, 128)	0
Conv2D (64 filters, 3×3)	(4, 4, 64)	73,792
BatchNormalization	(4, 4, 64)	256
MaxPool2D (2×2)	(2, 2, 64)	0
Flatten	(256)	0
Dense (512 units, ReLU)	(512)	131,584
Dropout (0.4)	(512)	0
Dense (24 units, Softmax)	(24)	12,312
Total params: ~402,834		

Dense ושתי שכבות Flatten ואחריהן, Max Pooling ו-Batch Normalization שכבות קונבולוציה עם 4 על 24 מחלקות Softmax פלט, ReLU הפעלה פנימית. Overfitting מונע (0.2–0.4) Dropout.

4. ארכיטקטורת המערכת והמתודולוגיה

4.1 AI-שילוב מודל ה

מאגר הנתונים

מאגר	גודל	תיאור
Sign Language MNIST (אימון)	תמונות 27,455	גווני אפור, 24 מחלקות 28×28 (A–Y ללא J,Z)
Sign Language MNIST (ולידציה)	תמונות 7,172	סט בדיקה נפרד
דגימות מותאמות (custom_train.csv)	כ-15–20 לאות (יעד)	נלכדו ממצלמת הפרויקט (capture_data.py)
הרחבת רקעים (bg_augmented_train.csv)	מרובה	הרכבת דגימות יד על רקעים מגוונים

תהליך האימון

- Adam אופטימיזצור, Batch Size 32, אימון מאפס — 20 אפוקים.
- סיבוב $\pm 35^\circ$, זום: ImageDataGenerator דרך (Data Augmentation) הרחבת נתונים $\pm 30\%$, הזזה $\pm 20\%$, בהירות 40%–160%, גזירה $\pm 25\%$, היפוך אופקי.
- לאחר איסוף דגימות מותאמות, המודל הקיים נטען ואומן מחדש — (Fine-tuning) כוונן עדין בקצב למידה נמוך (0.0001).
- ReduceLROnPlateau — 50% כשדיוק הולידציה לא משתפר במשך 2 אפוקים (עד מינימום 0.00001).

One-Hot ותוויות Softmax מתאימה לסיווג רב-מחלקתי עם, Categorical Cross-Entropy: פונקציית הפסד Hot.

קוונטיזציה והטמעה

כיוול על 500, (משקלות והפעלות כאחד) `int8` עם קוונטיזציה מלאה ל TensorFlow Lite-המרה ל לשמירת תאימות עם קוד `float32`-תמונות מייצגות מסט האימון, כשקלט/פלט נשארים ב `xxd` באמצעות `Header (model.h)` C++ מומר ל, `(.tflite)` גודל סופי: ~410KB-Inference. `-i`.

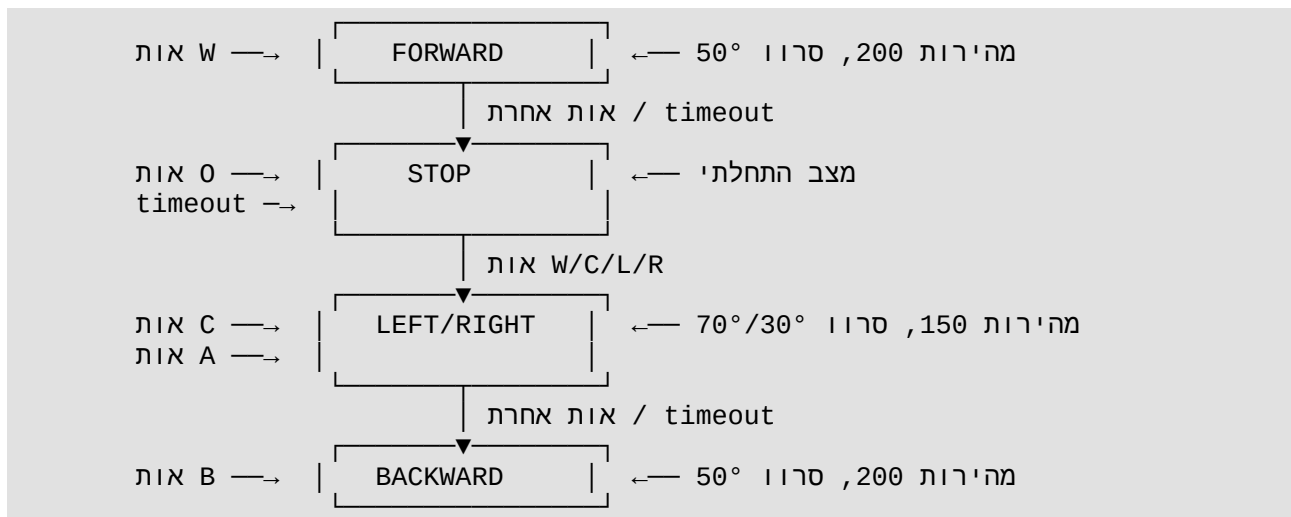
4.2 זרימת הנתונים (Pipeline)

```
graph TD
    A["A[<br>96x96 Grayscale"] -->|<br>70% מרכזי + Box-Average Downsample"| B["B[<br>עיובוד מקדים"]
    B -->|<br>28x28 normalized [0.0 - 1.0]| C["C[<br>CNN מודל TFLite int8 Quantized"]
    C -->|<br>Softmax, 40%≤ בודד D["D[<br>Majority Vote החלקה: על 5 פריימים"]
    D -->|<br>ממוצע E["E[<br>80%?> ביטחון"]
    E -->|<br>יכן F["F[<br>Serial1 TX → RX2"]
    E -->|<br>לא G["G[<br>התעלמות"]
    F --> H["H[<br>Arduino Mega פקודה"]
    H --> I["I[<br>W/C/L/R/S?> מיפוי אות"]
    I --> J["J[<br>50°> נסיעה קדימה"]
    I --> K["K[<br>50°> נסיעה אחורה"]
    I --> L["L[<br>70°> פנייה שמאלה"]
    I --> M["M[<br>30°> פנייה ימינה"]
    I --> N["N[<br>50°> עצירה"]
    I --> G["G[<br>אות אחרת"]
```

- לכידת תמונה — 96×96 גווני אפור, הגודל הקטן ביותר שהחומרה תומכת בו.
- עיובוד מקדים — חיתוך מרכזי (70% מהפריים) + דגימה-מחדש במוצע-תיבה ל- 28×28 , נורמליזציה ל- $[0.0, 1.0]$. אין חיתוך-יד אמיתי (יקר חישובית) — המשתמש ממרכז את היד ידנית.
- PSRAM ב-512KB של (Tensor Arena) נטען לזירת חישוב — (Inference) הרצת המודל, פלט: וקטור של 24 הסתברויות. TFLite Micro Interpreter מורץ דרך.
- סינון והחלקה — תחזית פריים-בודד נספרת רק אם ביטחונה $\leq 40\%$; אות נחשבת יציבה רק מה שמצמצם רעש. (Majority Vote) אם היא הרוב (3 מתוך 5) בחלון הפריימים האחרונים רק אז, אם ממוצע הביטחון שלה $\leq 80\%$, היא נשלחת לרכב.
- של (פין 17) RX2 אל, 9,600 baud, Serial1 (GPIO2), תקשורת — האות נשלחת כתו בודד דרך ה-Arduino Mega.
- ממפה את האות לפקודת תנועה ומפעיל מנועים וסרוו, עם Mega-הפעלת מנועים — ה לנסיעה ישרה (P-Controller) סנכרון מבוסס אינקודרים.

4.3 מערכת קבלת ההחלטות

פשוטה (State Machine) לוגיקת הרכב מבוססת מכונת מצבים:



מנגנוני בטיחות:

1. סף ביטחון (80%) — מסנן תחזיות לא ודאיות לפני שליחה לרכב.
2. של 1.5 שניות — אם לא מתקבלת פקודה חדשה, הרכב עוצר אוטומטית Timeout.
3. למניעת תנועות (2°Hz לאיטרציה ב-50) מעבר חלק בהיגוי — זווית הסרוו משתנה הדרגתית. פתאומית.

4.4 אפשרויות תקשורת

אפשרות 1 — חיבור ישיר בכבל (לרכב נייד):

ESP32-S3 Sense		Arduino Mega
D1 (GPIO2)	TX →	Pin 17 (RX2)
GND	→	GND

של USB-קורא את יציאת ה serial_bridge.py סקריפט: USB-אפשרות 2 — דרך מחשב ב Arduino של ה USB-מסנן לפי סף ביטחון, ומעביר את האות ליציאת ה, ה-ESP32 (115,200 baud) ה-Mega (9,600 baud).

```
python src/serial_bridge.py --esp32 COM3 --car COM5 --threshold 80
```

5. ניסויים ותוצאות

5.1 סביבת הניסוי

1. Docker עם מצלמת רשת בסביבת test_model.py — בדיקת מודל על המחשב.
2. קלט (simulator/index.html): בדיקת שרשרת מלאה — סימולטור מבוסס-דפדפן (Log Replay) ESP32-כתחליף לתנועות יד, הזנת לוגים אמיתיים מה (W/B/C/A/O) מקלדת Timeout-עם תזמון מקורי, הדמיה ויזואלית של הרכב, סינון ביטחון חי, וספירה לאחור של ה

5.2 תוצאות כמותיות

ESP32-דוגמה ללוג אמיתי מה

[105167 ms]	letter=G	confidence=98.8%	← לא פקודת רכב, מתעלם
[106398 ms]	letter=C	confidence=59.4%	← מתחת ל-80%, מתעלם
[107629 ms]	letter=G	confidence=37.9%	← מתחת ל-80%, מתעלם
[108860 ms]	letter=C	confidence=60.5%	← מתחת ל-80%, מתעלם
[113784 ms]	letter=X	confidence=80.1%	← עובר סף אבל לא פקודת רכב
[117477 ms]	letter=X	confidence=93.8%	← עובר סף אבל לא פקודת רכב

ממד	ערך
סה"כ תחזיות (בלוג לדוגמה)	19
תחזיות מעל 80%	3 (15.8%)
תחזיות מתחת ל-80%	16 (84.2%)
ביטחון ממוצע (כלל)	59.3%
ביטחון ממוצע (מעל סף)	90.9%

סף 80% מסנן ביעילות את רוב הרעש, ומעביר רק תחזיות בהן המודל בטוח ברמה גבוהה.

5.3 ביצועים בזמן אמת (Real-Time Performance)

ממד	ערך
מרווח בין תחזיות	~1,200 ms
Inference קצב	~0.8 FPS
צוואר בקבוק	לכידת פריים) על ESP32-S3 TFLite הרצת מודל (+ Invoke)
זמן תגובת רכב (מפקודה לתנועה)	< 50 ms (Arduino 50-בHz לולאת)
Baud Rate — רכב → ESP32	9,600 bps
Baud Rate — ESP32 USB (לוג)	115,200 bps

לשליטה ברכב זמן תגובה של ~1.2. ESP32-S3-מוגבל בכוח החישוב של ה FPS ניתוח: קצב ~0.8 שניות סביר — המשתמש מחזיק תנועת יד 1-2 שניות, ותוך זמן זה מתקבלת תחזית יציבה אחת אינו הכרחי למשימה זו, בניגוד לנהיגה אוטונומית (FPS כמו 30) לפחות. קצב עדכון גבוה יותר מהירה.

6. דיון, אתגרים ומסקנות

6.1 אתגרים והתמודדות

פתרון	אתגר
הקצאת tf.lm.esp32 הישיר של API-מעבר ל עם PSRAM-הזירה ידנית ב <code>heap_caps_malloc(..., MALLOC_CAP_SPIRAM)</code>	— (~255KB) פנימי מוגבל DRAM זיכרון הקצתה את זירת EloquentTinyML ספריית DRAM-פנימי וגרמה ל-DRAM-החישוב ב <code>segment data does not fit</code>
צינור תלת-שלבי: איסוף דגימות מותאמות הרחבת רקעים, (<code>capture_data.py</code>), כוונן עדין, (<code>augment_backgrounds.py</code>), בקצב למידה נמוך	המודל שאומן על — (Domain Shift) פער דומיין תמונות סטודיו הציג ביצועים גרועים על מצלמת הפרויקט
כיוול על 500 דגימות מייצגות, קלט/פלט נשאים המקורי Keras-הפרש הדיוק ממודל ה float32; מינורי	עלול לפגוע int8 — קוונטיזציה ואיכות מודל בדיוק
סף ביטחון, (Majority Vote) שכבת החלקה שניות, מעבר הדרגתי בהיגוי 3 Timeout, 80%	תגובה לפקודות לא ודאיות
בטוח (ESP32 TX → Mega RX) כיוון התקשורת ללא רכיבים נוספים; לכיוון ההפוך נדרש מחלק מתח	(V מול 3.3V 5) הפרש מתח

6.2 מסקנות

המערכת עומדת ביעדים שהוגדרו:

- ממופות לפקודות נסיעה ומופעלות בזמן (W/B/C/A/O) שליטה ברכב פיזי — חמש אותיות אמת.
- מונעים תגובות שגויות Timeout-סף ביטחון, Majority Vote — בטיחות.
- דרך מחשב USB גמישות חיבור — כבל ישיר או.

חוזקות: עצמאות מלאה לאחר ההצריבה (ללא מחשב/ענן); כוונן עדין מותאם לתנאי סביבה; בטיחות רבת-שכבות; סימולטור מקומי לבדיקה ללא חומרה.

שאינו מתאים למשימות הדורשות תגובה מיידית; אין חיתוך (~0.8 FPS) נמוך Inference חולשות: קצב אינן נתמכות (הדורשות תנועה) Z ו-J יד אוטומטי על המיקרו-בקר; האותיות

6.3 עבודה עתידית

- תמיכה מלאה ב ESP32S3.
- Z ו-J עבור (רצף פריימים / LSTM) זיהוי תנועות דינמיות.
- Inference-לפני ה (קל Skin Detection) חיתוך יד אוטומטי.

4. במקום כבל סריאלי (Bluetooth/WiFi) תקשורת אלחוטית.
5. מילון פקודות מורחב (מהירות, צופר, מצב חניה).
6. או ארכיטקטורה ייעודית למיקרו-בקרים MobileNetV2-Tiny — שיפור מודל.

7. נספחים ומקורות

7.1 קישורים לתוצרים

תוצר	קישור
מאגר קוד (GitHub)	github.com/LI3L/asl_training_model

7.2 מבנה קבצי הפרויקט

```

asl_training_model/
├── src/
│   ├── train_model.py          # אימון/כוונון המודל
│   ├── convert_model.py       # TFLite + model.h המרה ל
│   ├── test_model.py          # בדיקה עם מצלמה/תמונות סטטיות
│   ├── capture_data.py        # איסוף דגימות מותאמות
│   ├── capture_backgrounds.py  # לכידת תמונות רקע
│   ├── augment_backgrounds.py # הרכבת דגימות על רקעים
│   └── serial_bridge.py        # (אפשרות 2) USB גשר
├── deploy/
│   ├── esp32/ASL_Detector/     # ESP32-S3 סקיצת (שליחה לרכב + מודל + מצלמה)
│   └── arduino_mega/ASL_Car_Controller/ # Arduino Mega סקיצת (בקרת רכב)
├── simulator/
│   └── index.html              # סימולטור מקומי מבוסס דפדפן
├── data/                      # ותמונות CSV מאגרי נתונים
└── output/                    # (keras, tflite, model.h) מודלים מאומנים

```

7.3 מקורות

1. **Sign Language MNIST Dataset** — Tecperson, Kaggle.
<https://www.kaggle.com/datasets/datamunge/sign-language-mnist>
2. **TensorFlow Lite for Microcontrollers** — Google.
<https://www.tensorflow.org/lite/microcontrollers>
3. **tflm_esp32 Library** — Espressif / EloquentArduino, Arduino Library Manager.
4. **Seeed Studio XIAO ESP32S3 Sense** — תיעוד רשמי.
https://wiki.seeedstudio.com/xiao_esp32s3_getting_started/
5. **TensorFlow 2.15 Documentation** — Google. https://www.tensorflow.org/api_docs
6. **OpenCV Documentation** — OpenCV Team. <https://docs.opencv.org/>
7. **Arduino Mega 2560 Documentation** — Arduino.
<https://docs.arduino.cc/hardware/mega-2560>

8. **Keras ImageDataGenerator** — TensorFlow / Keras Documentation.
https://www.tensorflow.org/api_docs/python/tf/keras/preprocessing/image/ImageDataGenerator
9. **Post-Training Integer Quantization** — TensorFlow Guide.
https://www.tensorflow.org/lite/performance/post_training_integer_quant